

Final Project Design Brief

John Strange

Database

I spent a lot of time thinking about the structure of my database for this project. The most important part of the database is the relationship between Products and its specialized sub-tables. All tables such as Flies, Materials, and Equipment are derived from Products. This allows for scalability, as new product types can be added while still maintaining a shared structure.

Each order contains one or more OrderItems, and each OrderItem references a Product. This connects Orders and Products through OrderItems and stores things like quantity and checkout_price for each purchase. Each customer can have one or many orders.

Foreign Keys

- Orders.idCustomer $\rightarrow$ Customers.idCustomer
- OrderItems.idOrder $\rightarrow$ Orders.idOrder
- OrderItems.idProduct $\rightarrow$ Products.idProduct

Constraints

- OrderItems.quantity > 0
- Products.price > 0
- Products.stock ≥ 0
- OrderItems.checkout_price ≥ 0

Indexes

In some cases, we utilize indexes to quickly access tuples within the database.

- Orders.orderDate (used frequently for reporting queries)
- Products.idProduct (for quickly accessing product descriptions)

Front End

For the front end, I utilized a CSS framework called Bootstrap. This allows me to dynamically add styles through predefined CSS components. For example, when displaying products, I used the card component to define product images, pricing, and an add-to-cart button. Using Bootstrap helped ensure the UI components work cohesively.

For reporting, I used a JavaScript library called Chart.js. This allowed me to pull raw data from the database and display it in dynamically formatted charts. This approach can be extended to support different time intervals such as quarterly, biannual, annual, and weekly views, all loaded dynamically.

This approach could be extended further by storing aggregated reporting data in a data warehouse, allowing for precomputed summaries and reducing the computational overhead of repeated queries.

Back End

For the back end of this web application, I used Express along with the mysql2 library. Express allows me to define API endpoints that handle requests from the front end, while mysql2 is used to execute queries against the database.

I structured the server to use reusable generic query handling where possible. Some operations like checkout, require asynchronous handling to ensure that multiple database operations complete correctly.

The API returns results in JSON format, which integrates cleanly with the front end. In the future, I would implement transactions during the checkout process to make sure multiple checkouts don't interfere with each other, like inaccurate reading of product stock.